

Saralash algoritmlari

Saralash — bu berilgan sonlar ro'yxatini ma'lum tartibda (masalan, kichikdan kattaga qarab) qayta joylashtirish. Saralangan ro'yxatda kerakli sonni topish, eng katta yoki eng kichigini aniqlash, takrorlarni sezish — barchasi ancha osonlashadi. Shu sababli saralash — dasturlashning eng ko'p uchraydigan asosiy amallaridan biri.

Quyida eng sodda uch usulni ko'rib chiqamiz. Uchalasi bir xil natijaga olib keladi, ammo *fikrlash yo'li* har xil. Har birini bitta misolda — `[5, 2, 4, 1]` ro'yxatini o'sish tartibida saralash orqali — tushuntiramiz.

1. Pufaksimon saralash (bubble sort)

G'oyasi. Ro'yxat bo'ylab yurib, **qo'shni** ikki sonni solishtiramiz. Agar chapdagisi o'ngdagisidan katta bo'lsa, ularning o'rnini almashtiramiz. Bir marta to'liq yurib chiqilganda eng katta son ro'yxatning oxiriga "suzib" chiqadi. Buni ro'yxat to'liq tartiblanmaguncha takrorlaymiz. Nomi shundan: katta sonlar suvdagi pufakdek yuqoriga ko'tariladi.

Algoritm

1-usul. Tabiiy til (so'z bilan)

1. Ro'yxat a va uning uzunligi n ni olamiz.
2. Ro'yxat bo'ylab bir necha marta yuramiz (jami $n-1$ marta yetadi).
3. Har yurishda birinchi juftdan boshlab qo'shni sonlarni solishtiramiz: agar `a[j] > a[j+1]` bo'lsa, o'rnini almashtiramiz.
4. Barcha yurishlar tugagach, ro'yxat saralangan bo'ladi.

2-usul. Psevdokod

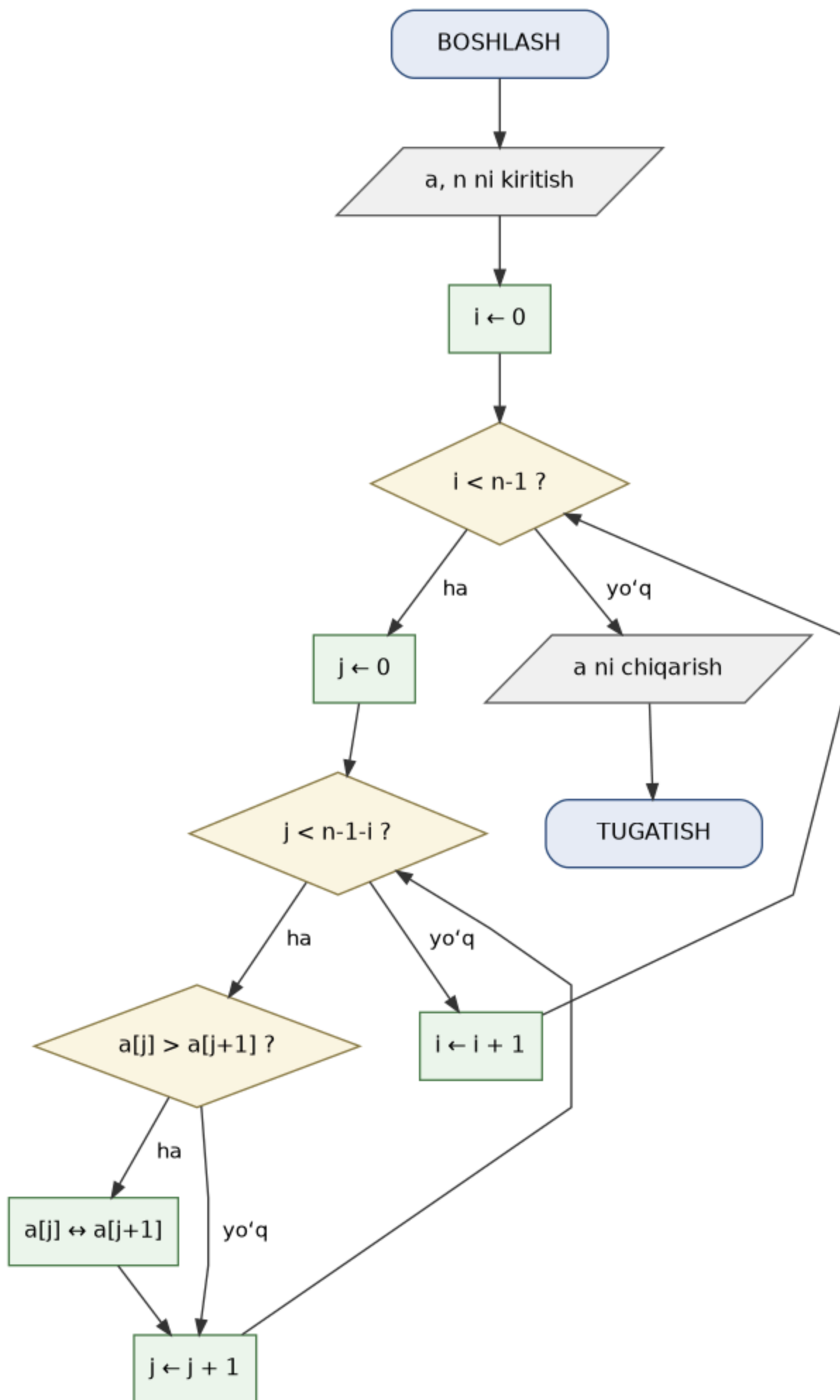
Algoritm: Pufaksimon(a , n)

Kirish: a ro'yxat, n uzunlik

Chiqish: o'sish tartibida saralangan a

1. $i \leftarrow 0$ dan $n-2$ gacha takrorla:
2. $j \leftarrow 0$ dan $n-2-i$ gacha takrorla:
3. agar $a[j] > a[j+1]$ bo'lsa:
4. $a[j]$ va $a[j+1]$ o'rnini almashtir
5. a ni qaytar

3-usul. Blok-sxema



Qadamlarni kuzatamiz — [5, 2, 4, 1] :

1-yurish: (5,2)→almash [2,5,4,1] → (5,4)→almash [2,4,5,1] →
(5,1)→almash [2,4,1,5]
2-yurish: (2,4) joyida → (4,1)→almash [2,1,4,5] → (4,5) joyida
3-yurish: (2,1)→almash [1,2,4,5]
Natija: [1, 2, 4, 5]

C++ kodi — avval faylga yozamiz

```
%%writefile pufaksimon.cpp
#include <iostream>
using namespace std;

// Pufaksimon saralash: qo'shni juftlarni solishtirib, almashtiramiz
void pufaksimon(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {           // nechta to'liq yurish
        for (int j = 0; j < n - 1 - i; j++) {   // qo'shni juftlar
            if (a[j] > a[j + 1]) {              // chapdagi kattaroqmi?
                int vaqt = a[j];                 // o'rnini almashtiramiz
                a[j] = a[j + 1];
                a[j + 1] = vaqt;
            }
        }
    }
}

int main() {
    int a[] = {5, 2, 4, 1};
    int n = 4;
    pufaksimon(a, n);
    cout << "Saralangan: ";
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Writing pufaksimon.cpp

Endi kompilyatsiya qilib ishga tushiramiz

```
!g++ pufaksimon.cpp -o pufaksimon.exe && pufaksimon.exe
```

Saralangan: 1 2 4 5

Python kodi

```
def pufaksimon(a):
    n = len(a)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a
```

```
a = [5, 2, 4, 1]
print("Saralangan:", pufaksimona(a))
```

Saralangan: [1, 2, 4, 5]

2. Tanlab saralash (selection sort)

G'oyasi. Ro'yxatni ikki qismga bo'lib tasavvur qilamiz: chapda saralangan qism, o'ngda saralanmagani. Har qadamda saralanmagan qismdan eng kichik sonni topamiz va uni o'sha qismning eng boshiga qo'yamiz. Shunda saralangan qism har qadamda bittaga uzayadi.

Algoritm

1-usul. So'z bilan

1. Ro'yxat a va uzunlik n ni olamiz.
2. Har bir i -o'rin uchun (0 dan $n-2$ gacha): i dan oxirigacha bo'lgan qismdan eng kichik sonning o'rnini topamiz.
3. Shu eng kichik sonni i -o'rindagi son bilan almashtiramiz.
4. Barcha o'rinlar to'lgach, ro'yxat saralangan bo'ladi.

2-usul. Psevdokod

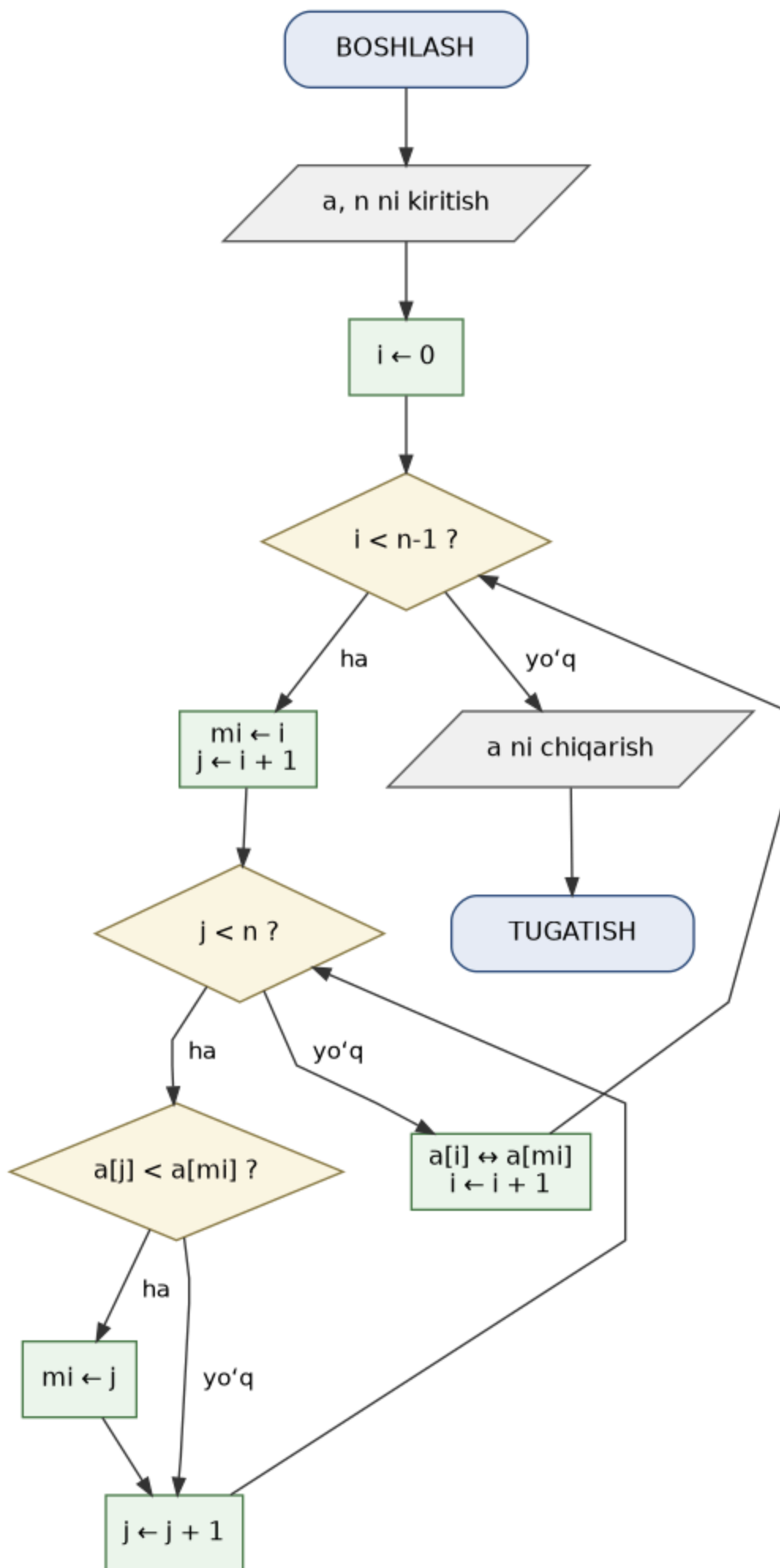
Algoritm: Tanlab(a , n)

Kirish: a ro'yxat, n uzunlik

Chiqish: o'sish tartibida saralangan a

1. $i \leftarrow 0$ dan $n-2$ gacha takrorla:
2. $mi \leftarrow i$ (eng kichigi indeks)
3. $j \leftarrow i+1$ dan $n-1$ gacha takrorla:
4. agar $a[j] < a[mi]$ bo'lsa:
5. $mi \leftarrow j$
6. $a[i]$ va $a[mi]$ o'rnini almashtir
7. a ni qaytar

3-usul. Blok-sxema



Qadamlarni kuzatamiz — [5, 2, 4, 1] :

1-qadam: [5,2,4,1] ichida eng kichik = 1 → 5 bilan almash → [1, 2, 4, 5]
2-qadam: [2,4,5] ichida eng kichik = 2 → joyida → [1, 2, 4, 5]
3-qadam: [4,5] ichida eng kichik = 4 → joyida → [1, 2, 4, 5]
Natija: [1, 2, 4, 5]

C++ kodi — avval faylga yozamiz

```
%%writefile tanlab.cpp
#include <iostream>
using namespace std;

// Tanlab saralash: har safar eng kichigini topib, oldinga qo'yamiz
void tanlab(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i; // eng kichigi shu deb faraz qilamiz
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min_idx]) { // undan kichigini topsak
                min_idx = j; // indeksini yodda tutamiz
            }
        }
        int vaqt = a[i]; // eng kichigini i-o'ringa qo'yamiz
        a[i] = a[min_idx];
        a[min_idx] = vaqt;
    }
}

int main() {
    int a[] = {5, 2, 4, 1};
    int n = 4;
    tanlab(a, n);
    cout << "Saralangan: ";
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Writing tanlab.cpp

Endi kompilyatsiya qilib ishga tushiramiz

```
!g++ tanlab.cpp -o tanlab.exe && tanlab.exe
```

Saralangan: 1 2 4 5

Python kodi

```
def tanlab(a):
    n = len(a)
    for i in range(n - 1):
        min_idx = i # eng kichigi shu deb faraz qilamiz
        for j in range(i + 1, n):
```

```
        if a[j] < a[min_idx]:           # undan kichigini topsak
            min_idx = j                 # indeksini yodda tutamiz
        a[i], a[min_idx] = a[min_idx], a[i] # eng kichigini oldinga qo'yamiz
    return a
```

```
a = [5, 2, 4, 1]
print("Saralangan:", tanlab(a))
```

Saralangan: [1, 2, 4, 5]

3. Joylashtirib saralash (insertion sort)

G'oyasi. Xuddi qo'lda karta tartiblashga o'xshaydi. Chapdagi saralangan qatorga o'ngdan bittalab son olib kelamiz va uni saralangan qismda **o'z o'rniga** suqib qo'yamiz: undan kattalarni bittaga o'ngga surib, bo'shagan joyga qo'yamiz.

Algoritm

1-usul. Tabiiy til (so'z bilan)

1. Ro'yxat a va uzunlik n ni olamiz. Birinchi son (bitta o'zi) saralangan hisoblanadi.
2. Ikkinchi sondan boshlab har bir sonni "kalit" deb olamiz.
3. Kalitdan chap tomondagi undan katta sonlarni bittaga o'ngga suramiz.
4. Bo'shagan joyga kalitni qo'yamiz.
5. Ro'yxat oxiriga yetganda, u to'liq saralangan bo'ladi.

2-usul. Psevdokod

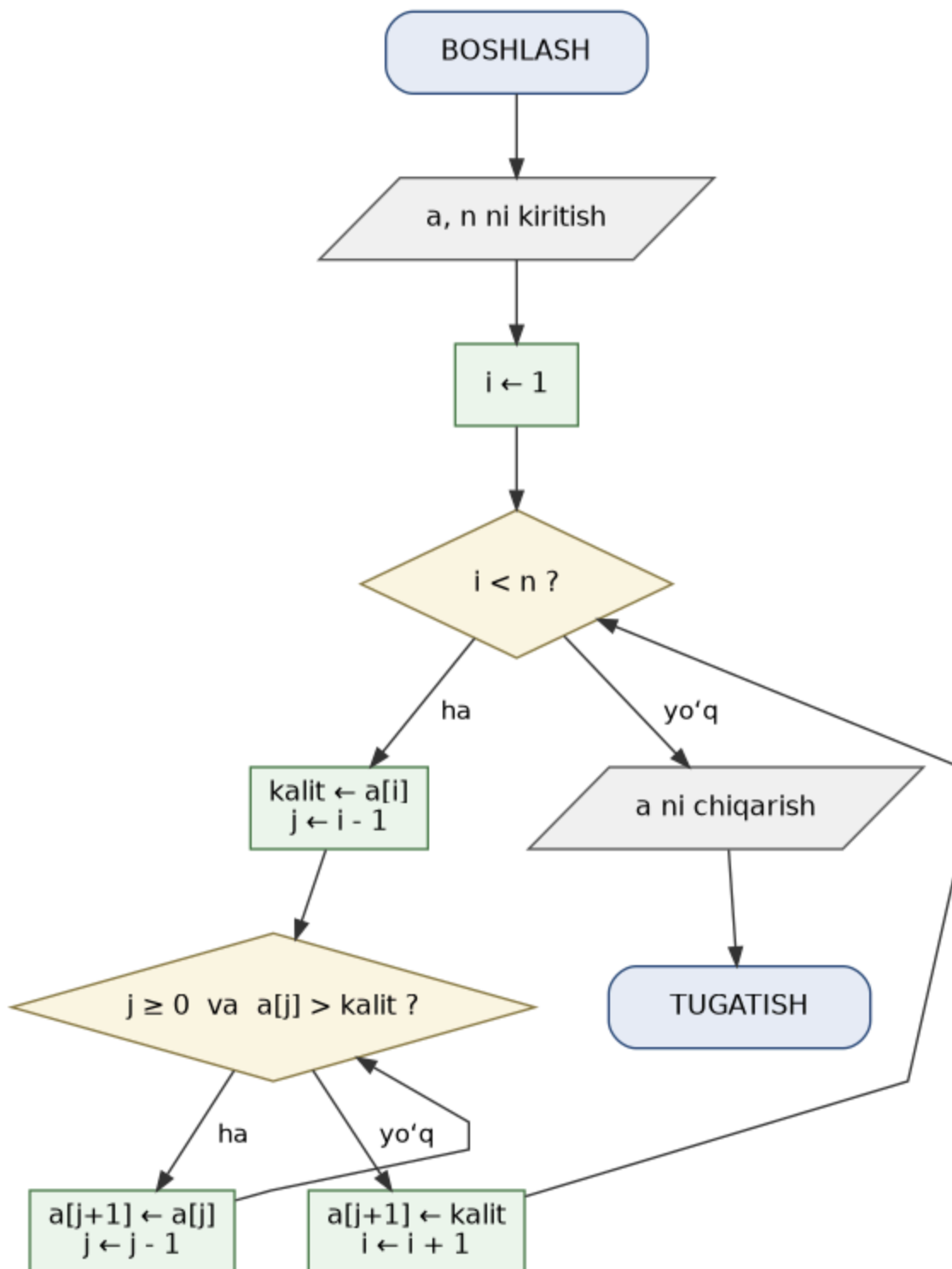
Algoritm: Joylashtirib(a , n)

Kirish: a ro'yxat, n uzunlik

Chiqish: o'sish tartibida saralangan a

1. $i \leftarrow 1$ dan $n-1$ gacha takrorla:
2. $kalit \leftarrow a[i]$
3. $j \leftarrow i-1$
4. $j \geq 0$ va $a[j] > kalit$ bo'lguncha takrorla:
5. $a[j+1] \leftarrow a[j]$ (o'ngga suramiz)
6. $j \leftarrow j-1$
7. $a[j+1] \leftarrow kalit$ (o'z o'rniga qo'yamiz)
8. a ni qaytar

3-usul. Blok-sxema



Qadamlarni kuzatamiz — `[5, 2, 4, 1]` :

Boshlang'ich: `[5 | 2, 4, 1]` (birinchi son saralangan)
 2 ni qo'yamiz: `2 < 5` → `[2, 5 | 4, 1]`
 4 ni qo'yamiz: `4 < 5, 4 > 2` → `[2, 4, 5 | 1]`
 1 ni qo'yamiz: `1 < 5, 4, 2` → `[1, 2, 4, 5]`
 Natija: `[1, 2, 4, 5]`

C++ kodi — avval faylga yozamiz


```

%%writefile joylashtirib.cpp
#include <iostream>
using namespace std;

// # Joylashtirib saralash: har bir sonni saralangan qismga o'z o'rniga qo'yamiz
void joylashtirib(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int kalit = a[i];           // joylashtiriladigan son
        int j = i - 1;
        // # kalitdan katta bo'lganlarni bittaga o'ngga suramiz
        while (j >= 0 && a[j] > kalit) {
            a[j + 1] = a[j];
            j = j - 1;
        }
        a[j + 1] = kalit;           // # kalitni bo'shagan joyga qo'yamiz
    }
}

int main() {
    int a[] = {5, 2, 4, 1};
    int n = 4;
    joylashtirib(a, n);
    cout << "Saralangan: ";
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
    return 0;
}

```

Writing joylashtirib.cpp

Endi kompilyatsiya qilib ishga tushiramiz

```
!g++ joylashtirib.cpp -o joylashtirib.exe && joylashtirib.exe
```

Saralangan: 1 2 4 5

Python kodi

```

def joylashtirib(a):
    n = len(a)
    for i in range(1, n):
        kalit = a[i]           # joylashtiriladigan son
        j = i - 1
        # kalitdan katta bo'lganlarni bittaga o'ngga suramiz
        while j >= 0 and a[j] > kalit:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = kalit       # kalitni bo'shagan joyga qo'yamiz
    return a

a = [5, 2, 4, 1]
print("Saralangan:", joylashtirib(a))

```

Saralangan: [1, 2, 4, 5]

Uchala usulni taqqoslash

Uchala algoritm ham bir xil saralangan ro'yxatni beradi, farqi — ishlash uslubida:

- **pufaksimom** — qo'shnilarni qayta-qayta almashtiradi; tushunish eng oson, lekin ko'p almashtirish qiladi;
- **tanlab** — har safar eng kichigini qidiradi va oldinga qo'yadi; almashtirishlar soni kam;
- **joylashtirib** — har bir sonni tartibli qismga suqib qo'yadi; ro'yxat deyarli tartibli bo'lsa juda tez ishlaydi.

Uchalasi ham kichik ro'yxatlar uchun juda qulay. Ro'yxat kattalashganda (masalan minglab son) ular sekinlashadi — bunday holatlar uchun tezroq algoritmlar bor, lekin ularning g'oyasini tushunish uchun avval mana shu uch sodda usulni bilish kerak.